`mabel_hw`: the MABEL Hardware API

A typed, write-leased, calibration-aware library under the single HAL

MABEL Project — Hardware & Control

November 2, 2026

Abstract

`mabel_hw` is the installable Python package the MABEL hardware bridge is built from — and the API any in-process consumer programs against. It layers the proven per-device drivers (MAXSwerve UDP, DaMiao MIT over Teensy-Ethernet or CAN, ORCA Feetech serial, the lift Pico, the RPLIDAR) under one facade, `MabelHardware`, with a frozen typed state contract (`RobotState`), *many-reader / single-writer* semantics enforced by per-actuator-group write leases, a unified as-deployed calibration store (`calibration_data/`), and graceful degradation everywhere. The DDS topics, video ports, and the `:8779` hand RPC are thin adapters over this same object, byte-compatible with the pre-refactor bridge. We quantify the layer's costs: a lease cycle is 5.5 µs (p50), the facade adds ∼17 µs to the base command path, a full-robot snapshot costs 134 µs — after an experiment-driven fix that memoizes the neck's RSSR forward kinematics (a 43× regression the harness caught).

Keywords: hardware abstraction, robot middleware, write arbitration, calibration management, teleoperation.

1 Motivation: from monolithic HAL to library + adapters

The bridge (`docs/hardware_bridge.pdf`) established the ONE-process rule: a single OS process owns every device, ROS and the apps are thin clients. What it lacked was an *importable seam*: every consumer — ROS nodes, the teleop server, control GUIs, experiment scripts — spoke raw DDS topics, the driver cores were scattered across ad-hoc folders, the DaMiao MIT driver existed in triplicate across the repo, and calibration lived in four locations with no single reader. `mabel_hw` keeps the process model and the wire contract *identical* while reorganizing the inside into the library pattern already proven by `controller/mabel`: an installable package, a facade, typed contracts, back-compat shims guarded by tests.

2 Package layout and the import contract

layer	modules
facade	`api.py` (`MabelHardware`, handles, writers)
contracts	`types.py` (frozen dataclasses), `errors.py`
arbitration	`arbitration.py` (`LeaseBroker`/`Lease`)
calibration	`calibration/` (store + anchor math)
subsystems	`subsystems/` (typed, calibrated, ROS-free cores)
drivers	`drivers/` (protocol only: swerve, teensy_link, damiao_can, orcahand, lift, rplidar, robot_host [vendored])
kinematics	`kinematics/neck.py` (RSSR ik/fk over neck_studio)
adapters	`ros/` (DDS nodes + the bridge supervisor), `rpc/` (:8779), `cameras/` (per-process video plane)

Import weight is tiered: `import mabel_hw` is lazy (PEP 562) and pulls no heavy dependency; `rclpy` is legal *only* under `mabel_hw.ros` and `mabel_hw.cameras` (enforced by an import-hook test). Four spawn/import surfaces are frozen for the outside world: the launcher file `hardware_bridge.py`, `python3 -m hands.hand_bridge`, `python3 -m cameras.camera_worker`/`video_relay` (shims), and the cwd-anchored top-level names (`config`, `base`, `hands`, `subsystems`, `cameras`) — every pre-refactor path is a pure re-export shim whose object identity is asserted in `tests/test_shims_identity.py`.

3 The facade

```
from mabel_hw import MabelHardware

with MabelHardware() as hw:                       # flock -> calibration -> connect
    state = hw.read()                             # RobotState frozen snapshot
    with hw.left_arm.claim("teleop") as arm:      # exclusive writer for the group
        arm.set_joint_positions({"l_j1": 0.3}) # model-frame radians
    hw.estop()                                    # lease-free, always allowed
```

`read()` returns a frozen `RobotState` whose per-subsystem fields (`left_arm`, `right_arm`, `neck`, `torso`, `base`, `lift`, `left_hand`, `right_hand`, `sensors`) are `None` whenever that subsystem is disabled or degraded — the snapshot itself always succeeds. All units are SI (rad, m, N m, °C); stamps are host-monotonic with `.age()`. `subscribe(cb, rate_hz)` pushes snapshots on a dedicated thread; any number of subscriptions coexist. Handles expose read surfaces freely (`state()`, `health()`, `on_state()`); *every* mutating method exists only on a Writer obtained via `handle.claim()`. Inside the bridge, node timers pace the drivers; a standalone consumer calls `hw.pace()` once.

4 Arbitration: exactly one writer per actuator group

Single-writer is enforced at three scopes:

1. **process** — the flock singleton (`/tmp/mabel_hardware_bridge.lock`): one device-owning process per host, with stale-owner recovery and systemd standby;
2. **writer** — `LeaseBroker`: within that process, at most one live `Lease` per group (`base`, `left_arm`, `right_arm`, `neck`, `torso`, `lift`, `left_hand`, `right_hand`). `claim(wait=)` blocks, `steal=True` takes over loudly, `ttl=` auto-expires an idle lease so a crashed writer cannot wedge its group. A stale writer's next write raises `NotLeaseHolder` — the command never reaches hardware;

3. **DDS** — every command-consuming ROS node runs a 2 s publisher-count watchdog and logs ERROR on > 1 publishers: the historical "`/mabel_cmd` two publishers → the base won't move" failure, now detected instead of silent.

estop() and stop() never require a lease. Safety cannot be write-locked: base estop latches zero drive; joint groups and hands hold-in-place (torque is *not* dropped — releasing an arm under load is a deliberate, leased act).

Measured (fig. 1, n=5000): an uncontended claim+check+release cycle costs 5.5 µs p50 / 7.5 µs p99; a contended handover (release wakes a blocked claimant) 61 µs p50 / 0.6 ms p99 — irrelevant at the 50 Hz command rates it guards.

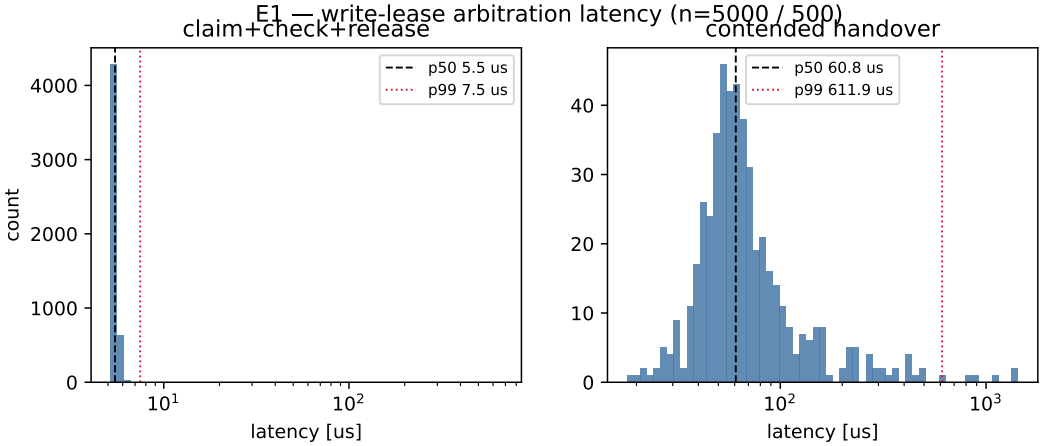


Figure 1: E1 — write-lease latency: uncontended cycle (left), contended handover (right). Log-x histograms, p50/p99 marked.

5 Drivers and the backend seam

Arms, neck, and torso implement one interface (`drivers.seam.JointGroupDriver`: latch `set_targets()`, re-send on `tick()` to feed firmware watchdogs, cached `read()`), with three backends selected per-subsystem by config:

backend	transport	used by	notes
`teensy`	UDP protobuf :5800, ≤1 kHz (vendored upstream `robot_host`)	arms, neck (target)	self-describing RobotInfo; **connect() enables motors** (damping hold); silence → limp
`can`	socketcan DaMiao MIT	torso; arms/body legacy	today's proven path
`mock`	none	tests, `mock=True`	deterministic first-order plant

One `TeensyLink` per physical Teensy is registry-shared: the openArm board carries both arms, so `left_arm` and `right_arm` are *views* whose slice-writes merge into a single full-length MIT vector per send (the begin/commit batching). Torque-off reverts a slice to the firmware-style damping hold and keeps streaming, so "off" is immediate rather than a 150 ms watchdog tail. The base keeps the validated ps5 path (`SwerveBase`: twist → host IK → calibration → Teensy), now with typed per-module state (steer/velocity/position/torque-estimate) and electrical health (V/A/°C/duty).

Hot-replug (every backend, no bridge restart). All drivers auto-reconnect and the facade announces membership: a dying CAN socket marks its bus `dead` (the RX thread exits cleanly, sends fail fast) and `DamiaoGroup.service()` rebuilds it — fresh socket, re-add, re-enable (respecting the torque state) — the moment the channel reopens; per-joint feedback ages expose even a single silent motor. A Teensy group may exist *before* its board answers (unresolved: reads dead, writes latch) and attaches via `service()` within ∼3 s of the board appearing; a board that reboots mid-run rejoins **LIMP** — latched targets cleared, torque off — so it can never snap to stale commands (streaming teleop re-commands within one cycle). The facade's 1 Hz presence watcher logs "`<name> LEFT the system`"/"`REJOINED`" transitions and `hw.presence()` returns the live membership map. Covered by `tests/test_hot_replug.py` (dead-bus rebuild, rate limiting, torque-state preservation, hot-join, limp-on-rejoin, presence transitions). Hands, lift, and lidar wrap the proven ORCA, Pico, and RPLIDAR drivers; cameras remain process-per-camera with the API exposing a stream *directory* (URLs + liveness), never frames — an in-process frame path would resurrect the GIL contention the split fixed.

6 The neck: task space over the RSSR linkage

`kinematics/neck.py` ports the upstream bridge kinematics over the canonical `hardware/neck_studio` closed-form model (never vendored; path bootstrapped, `MABEL_NECK_STUDIO` override). `NeckWriter.set_gaze (pitch, roll, yaw)` clamps to the calibrated `_task` limits, runs the linkage IK (common-mode cranks = pitch, differential = roll, direct-drive yaw), and raises `InfeasiblePose` at the model's feasibility wall (which includes the CS10 ball-swing cone — the real ±14.5° limiter). Feedback FK is a damped multi-seeded Newton with a linearized fallback so the task readout degrades to *approximate*, never dark. Without the studio checkout everything degrades to motor space with one loud warning.

FK memoization (found by E2). The first implementation ran the Newton FK on every `read()`: 5.7 ms/snapshot, collapsing the full-robot read rate to 174 Hz. FK is now memoized on the crank angles (new feedback → one solve), restoring 134 µs (fig. 2) — a 43× recovery the experiment harness caught before the robot ever did.

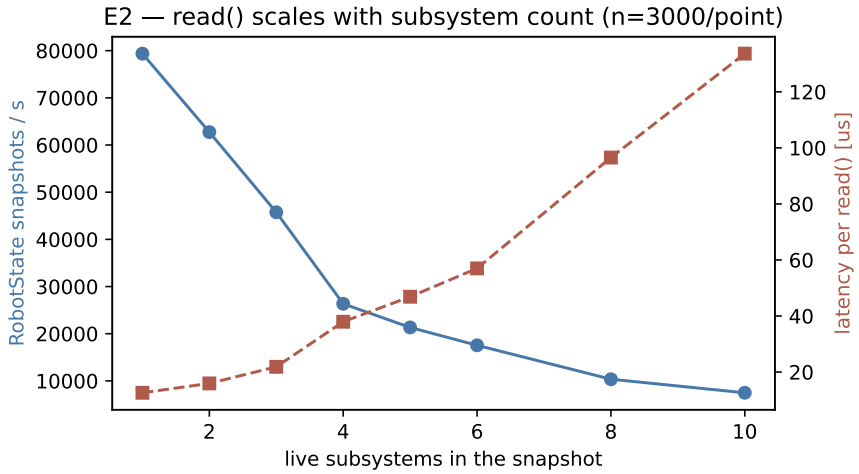


Figure 2: E2 — `read()` cost vs. live subsystems (mock plants, n=3000/point), after the neck-FK memoization.

7 Calibration: one folder, dual-written, hot-reloaded

`calibration_data/` is the bridge's read-first mirror: `mabel.json` (exact copy of the upstream studio tree — `arm/openarm-dual`, `base/maxswerve`, `hand/orca-*`, `neck/rssr`) plus `hands/<side>_calibration.json` tick clamps. Writers dual-write on every save (the hand RPC's `save_calib` fan-out, the firmware GUI, the upstream calib server via `MABEL_CALIB_MIRRORS`); `tools/sync_calibration.py` is the pull-based fallback; the `CalibrationStore` falls back to the legacy locations when the mirror is absent, so nothing is ever fatal. Anchor semantics are preserved per family and verified by round-trip tests:

$$\text{center (arm/hand):}\quad q = q_{mid}^{model} + d\,(r - \tfrac{r_{min}+r_{max}}{2}), \qquad \text{neutral (neck):}\quad q = d\,(r - r_{neutral}),$$

with $d = \pm 1$ and `to_raw` clamped to the measured end-stops in both families. The store `watch()`es its resolved sources; a studio save re-derives every joint map atomically — targets are latched in model frame, so a calibration swap re-rasterizes on the next tick without a motion glitch beyond the calibration delta itself.

8 ROS adapters: same wire, leased inside

The six nodes (`mabel_hw/ros/*_node.py`) keep the DDS contract byte-identical — topics, message types, latched torque-state QoS — and each accepts `build(cfg, hw=)`: with the bridge's shared facade they command through write-leased handles; standalone (or with the new-taxonomy groups disabled, today's default) they run the legacy driver paths unchanged. Two translation boundaries live in the adapters, not the API: the URDF arm names (`left_arm_N` ↔ upstream `l_jN`) and the ORCA thumb ↔ URDF rename. The body node maps the DDS task-space neck names onto `set_gaze()` and publishes FK-derived task names, so `/body/*` survives the mechanism change from direct-drive to RSSR. Measured on the fake-Teensy loopback (fig. 3, n=3000): the leased facade path costs 104 µs p50 vs. 87 µs direct — 17 µs for the lease check, typed layer, and immediate push, i.e. 0.1% of a 50 Hz period.

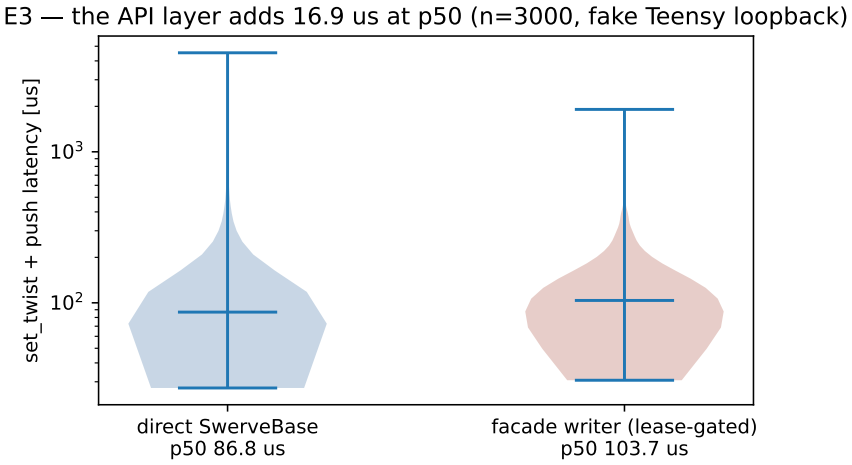


Figure 3: E3 — base command path: direct `SwerveBase` vs. the lease-gated facade writer, against the wire-exact fake Teensy.

9 Testing

Everything below the ROS layer runs hardware-free on any machine (`python3 -m pytest tests -q`; 156 tests): the wire-exact fakes (`tools/fake_teensy.py` for the base, the vendored upstream `MockTeensy` for arms/neck, `MockFeetechBus` hands, a stubbed lift, python-can `virtual` buses for the MIT framing), lease semantics (64-thread hammering), the calibration store (precedence/fallback/anchor round-trips/hot reload), the full facade on mock plants, shim identity, the frozen contract snapshot (`tests/contract/contract.json`), and an AST-level structure lint of the ROS nodes. On thor the `ros`-marked tests execute for real and `tools/software_selftest.sh` (unchanged) remains the end-to-end acceptance gate — always bracketed by stopping/starting the systemd unit, since it kills any running bridge.

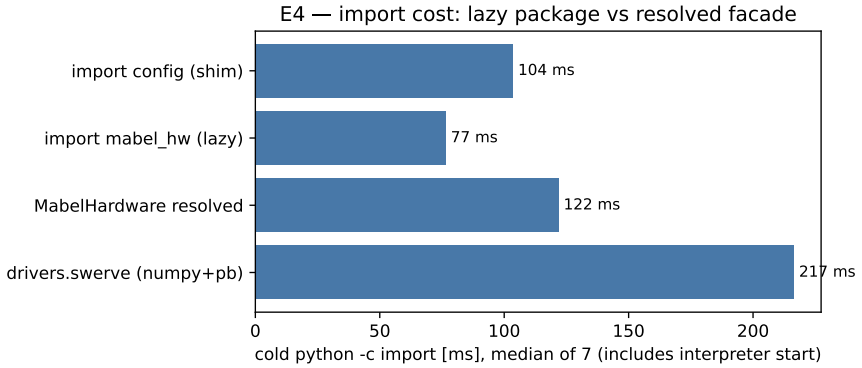


Figure 4: E4 — cold-import cost (subprocess median of 7; includes interpreter start ≈ 45 ms): the lazy package stays cheap; heavy deps load only when a driver is touched.

10 Experiments summary

All regenerated by `python3 experiments/exp_hw_api.py` (hardware-free).

metric	value	
E1 lease claim+check+release p50	5.5	µs
E1 lease cycle p99	7.5	µs
E1 contended handover p50	60.8	µs
E2 full-robot `read()` (10 subsystems)	133.6	µs
E2 full-robot snapshot rate	7483	Hz
E3 facade overhead on the base path (p50)	16.9	µs
E4 `import mabel_hw` (cold, median)	76.6	ms

11 Limitations and open items

- The Teensy-Ethernet arm/neck backends are validated against the wire-exact mock only; the real openArm Teensy bench check (Hello/RobotInfo against actual firmware, first motion) is a user-run checklist item, and the neck hardware itself is not built yet.
- E2/E3 numbers are Mac-measured; thor (Jetson) will differ by a constant factor — rerun the harness there when convenient (it needs no hardware).

- The orcahand driver still exists as three code copies across the repo (firmware, this package, upstream); this refactor unified only the *data* (calibration). Code convergence is future work.
- Task-space neck feedback publishes zero velocities/efforts (task-rate Jacobians are not derived); consumers needing motor rates read the `NeckState.motors` block.
- The in-process camera surface is deliberately URL/liveness only.

12 References

- `docs/hardware_bridge.pdf` — the process model, base-path validation, drive under-reach, and safety history this API inherits.
- `controller/docs/mabel_controller.pdf` — the `mabel` library pattern (facade / seam / shims) replicated here.
- upstream `modules/arm/firmware/docs/HOST_PROTOCOL.md` — the Teensy wire protocol (vendored client at `drivers/robot_host`).
- `hardware/neck_studio` — the canonical RSSR model + validation.
- `calibration_data/README.md` — the dual-write topology.